1) Create a 10×10 array of random integers (1–100). Replace all even numbers with -1. Extract the diagonal without using np.diag.

2) Create a 20×20 checkerboard pattern (0s and 1s) using slicing (no loops).

3) Given arr = np.random.randint(1, 100, (10,10)), find numbers greater than row mean, and replace elements divisible by 7 with negative.

4) Extract all prime numbers from 1D array of size 1000.

5) Create 10×10 arrays where A[i,j] = i + j and B[i,j] = i * j without loops.

6) Generate a random 5×5 matrix. Compute determinant, rank, trace, eigenvalues, and verify A @ A^{-1} = I.

7) Solve 3x + 2y – z = 1
2x – 2y + 4z = –2
–x + ½y – z = 0

```
#1
import numpy as np

checkerboard = np.zeros((20, 20), dtype=int)
checkerboard[::2, ::2] = 1
checkerboard[1::2, 1::2] = 1
display(checkerboard)
```

```
array([[1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
       [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
       [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]])
```

```
#3
import numpy as np
arr = np.random.randint(1, 100, (10, 10))
```

```
display("Original array:", arr)

row_means = arr.mean(axis=1, keepdims=True)
greater_than_mean = arr > row_means
display("Elements greater than row mean:", greater_than_mean)

arr[arr % 7 == 0] *= -1
display("Array with elements divisible by 7 replaced with negative:",
arr)
```

```
{"type":"string"}
```

```
array([[22, 69, 68, 75, 97,  7, 99, 36, 44, 40],
       [68, 74, 26, 55, 67, 33, 91, 73, 80, 74],
       [ 2,  7, 85, 88, 47, 55, 78, 88,  5, 54],
       [ 7, 75, 40, 50, 86, 51, 52, 33, 48, 17],
       [27, 54, 69, 23, 64, 93, 20, 40, 16, 41],
       [36, 42, 98, 95, 89, 76, 78, 38, 68, 15],
       [24,  1, 44, 42, 41, 51, 70, 75, 50, 65],
       [89, 67, 41, 59, 57, 43, 16, 41, 12, 12],
       [38, 67, 18, 69, 12, 98, 51, 10, 94, 22],
       [85, 18, 43, 13,  2, 63, 27, 40, 37, 91]])
```

```
{"type":"string"}
```

```
array([[False,  True,  True,  True,  True, False,  True, False, False,
        False],
       [ True,  True, False, False,  True, False,  True,  True,  True,
         True],
       [False, False,  True,  True, False,  True,  True,  True, False,
         True],
       [False,  True, False,  True,  True,  True,  True, False,  True,
        False],
       [False,  True,  True, False,  True,  True, False, False, False,
        False],
       [False, False,  True,  True,  True,  True,  True, False,  True,
        False],
       [False, False, False, False, False,  True,  True,  True,  True,
         True],
       [ True,  True, False,  True,  True, False, False, False, False,
        False],
       [False,  True, False,  True, False,  True,  True, False,  True,
        False],
       [ True, False,  True, False, False,  True, False, False, False,
         True]])
```

```
{"type":"string"}
```

```
array([[ 22,  69,  68,  75,  97,  -7,  99,  36,  44,  40],
       [ 68,  74,  26,  55,  67,  33, -91,  73,  80,  74],
       [  2,  -7,  85,  88,  47,  55,  78,  88,   5,  54],
```

```
       [ -7,  75,  40,  50,  86,  51,  52,  33,  48,  17],
       [ 27,  54,  69,  23,  64,  93,  20,  40,  16,  41],
       [ 36, -42, -98,  95,  89,  76,  78,  38,  68,  15],
       [ 24,   1,  44, -42,  41,  51, -70,  75,  50,  65],
       [ 89,  67,  41,  59,  57,  43,  16,  41,  12,  12],
       [ 38,  67,  18,  69,  12, -98,  51,  10,  94,  22],
       [ 85,  18,  43,  13,   2, -63,  27,  40,  37, -91]])
```

```
#4
def is_prime(n):
    """Checks if a number is prime."""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

data = np.arange(1, 1001)
prime_numbers = data[np.vectorize(is_prime)(data)]
display(prime_numbers)
```

```
array([  2,   3,   5,   7,  11,  13,  17,  19,  23,  29,  31,  37,
41,
        43,  47,  53,  59,  61,  67,  71,  73,  79,  83,  89,  97,
101,
       103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163,
167,
       173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233,
239,
       241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311,
313,
       317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389,
397,
       401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463,
467,
       479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563,
569,
       571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641,
643,
       647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727,
733,
       739, 743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821,
823,
       827, 829, 839, 853, 857, 859, 863, 877, 881, 883, 887, 907,
911,
       919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997])
```

```
#7
import numpy as np
```

```python
A = np.array([[3, 2, -1],
              [2, -2, 4],
              [-1, 0.5, -1]])

b = np.array([1, -2, 0])

solution = np.linalg.solve(A, b)

print("The solution is:")
print(f"x = {solution[0]}")
print(f"y = {solution[1]}")
print(f"z = {solution[2]}")
```

```
The solution is:
x = 0.9999999999999994
y = -1.9999999999999984
z = -1.9999999999999984
```